

Computer Systems and Networks

Lab 2 — Part B, Task 3

Is3 Dashboard — Architecture Document

Team — M26SE01

Kirill Arkhipov

Evgeny Anisov

Gabdrakhmanov Rasil

Ivanov Sergey

Supervisor & Instructor: dr. Paolo Ciancarini

TAs: Daneel Leontiev

Vasily Ziakbin

1 Scope

Is3 is a near-real-time dashboard that reports the status of a customer's registered IoT devices — cameras, electric switches, fridges, thermostats and intercoms — on a single screen. Three constraints from the brief shape every decision in this document:

1. The data is **stateless status** pushed by devices in the field. The system presents it; it does not own it.
2. The customer **never writes**. No create, no update, no delete.
3. There is **no registration**. Sales onboards customers manually and the devices are already registered when the user first signs in.

These remove most of the usual complexity of an IoT platform: command and control, device provisioning, write conflicts and user-generated content are all out of scope. The architecture below exploits that rather than ignoring it, and the single most important consequence appears in every section: *with no user writes, the read path can be replicated and cached aggressively without any consistency problem worth the name.*

This document is the deliverable for step 3.5: it defines the requirements (3.1), maps the components (3.2), justifies the technology stack (3.3) and presents the architecture design (3.4).

2 Architectural goals

1. A device status change reaches the technician's screen within 5 seconds.
2. Ingest load never degrades the display; the two fail independently.
3. The system holds 10 000 devices per customer without a redesign.
4. A customer can never see another customer's device.
5. The technician can tell which devices need attention at a glance, without reading every row.

Goals 1–3 are architectural in the strict sense: they are expensive to retrofit and they decide the shape of the system. Goal 4 is a constraint on every component that touches data. Goal 5 is why the interface looks the way it does, and it is the goal most easily lost when a dashboard grows.

3 Requirements (3.1)

a. Functional

ID	Requirement	Priority
FR-1	Ingest status messages from registered devices in the field	Must
FR-2	Reject messages from any device not registered to a customer	Must
FR-3	Show every device belonging to the signed-in customer on one screen	Must
FR-4	Show each device's status, kind, site and last-seen time	Must
FR-5	Refresh the view in near real time without a manual reload	Must
FR-6	Mark a device offline when no message arrives within its expected interval	Must
FR-7	Execute a fixed set of pre-defined queries chosen by the customer	Must
FR-8	Filter the device view by site, device kind and status	Must
FR-9	Show a single device's recent history on demand	Must
FR-10	Show fleet-level summaries: counts by status, uptime over time	Should
FR-11	Authenticate the customer and scope all data to their tenant	Must
FR-12	Record an audit trail of who viewed what	Should
FR-13	Export the current view as CSV	Could

Deliberately excluded: device registration, device control, customer self-service sign-up, free-form query authoring, editing any device data.

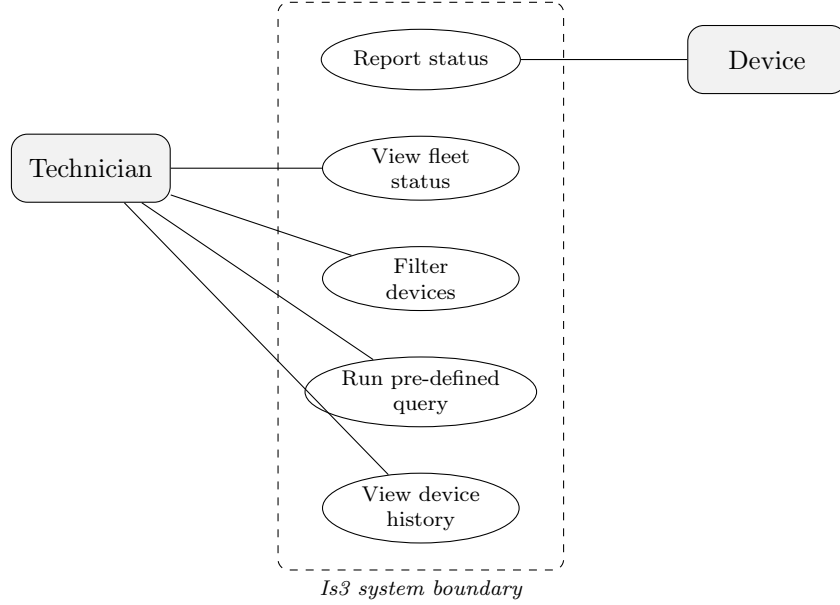


Figure 1: Use cases. Every technician case is a read. The only writer in the system is the device itself, and it writes status, never state the customer chose. There is deliberately no “configure device” or “edit” case.

b. Non-functional

ID	Attribute	Requirement	Measured by
NFR-1	Performance	Status change visible within 5 s (p95)	trace timestamp
NFR-2	Performance	First contentful paint under 1.5 s on 4G	Lighthouse
NFR-3	Scalability	10 000 devices/customer, 50 000 msg/min	load test
NFR-4	Scalability	Interactive with 1 000 devices rendered	frame budget
NFR-5	Availability	99.9 % monthly on the read path	uptime monitor
NFR-6	Reliability	One ingest node lost loses no acknowledged message	chaos test
NFR-7	Security	Per-device credential; transport is TLS	pen test
NFR-8	Security	Tenant isolation across every endpoint	automated authz test
NFR-9	Usability	Technician spots devices needing attention within 5 s	usability test
NFR-10	Accessibility	WCAG 2.1 AA; status never by colour alone	axe + keyboard pass
NFR-11	Maintainability	Components independently testable; no shared mutable state	code review
NFR-12	Portability	Frontend is a static bundle for any CDN	static-host deploy test
NFR-13	Observability	Every pipeline stage emits metrics and traces	dashboards exist
NFR-14	Cost	Read path cacheable; cost sublinear in traffic	cost per 1 000 sessions

Driving quality attributes

Three dominate and they conflict usefully. Near-real-time latency (NFR-1) pushes toward streaming and push transport. Read scalability (NFR-3, NFR-4) pushes toward caching and pre-aggregation. Availability (NFR-5) pushes toward decoupling ingest from display. The read-only constraint is what lets all three hold at once.

4 Key abstractions

Five concepts carry the whole domain. Everything else in the system is machinery around them.

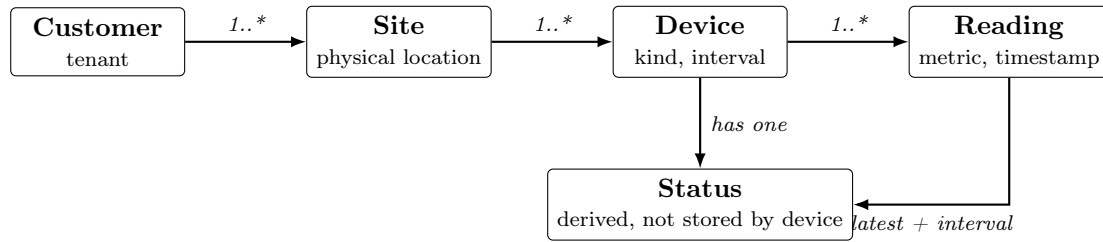


Figure 2: Domain model. **Status** is the only derived concept: it is computed from the latest reading and the device’s expected interval, never sent by the device. That is what makes silence detectable — a device that stops reporting cannot declare itself offline.

5 Component map (3.2)

Ingest is write-heavy and bursty; reads are repetitive over a small hot set. Because customers never write, the two paths never contend for the same data, so we separate them and let each scale on its own terms.

Group	Component	Responsibility
Ingest	Device Gateway	Terminate MQTT/TLS, authenticate each device
	Registry Check	Drop messages from unregistered devices (FR-2)
	Ingest Stream	Durable buffer between gateway and processing
Processing	Status Processor	Derive current status, write it to the state store
	Staleness Monitor	Flip a device to offline on silence (FR-6)
	Aggregator	Maintain per-customer rollups
Storage	Current State	One row per device: latest status and reading
	Time-Series	Historical readings with retention
	Registry DB	Customers, sites, devices, entitlements
Serving	Read API	Device lists, single-device history, queries
	Query Catalogue	The fixed pre-defined query set (FR-7)
	Push Service	Stream updates to open dashboards (FR-5)
	Auth Service	Tenant-scoped token (FR-11)
Client	Dashboard SPA	The static frontend
	Feed Client	Holds the push connection
	View State	Filters and selection, client-only

Two components deserve a note. The **Ingest Stream** is the seam: if processing dies, devices keep publishing and nothing is lost. The **Staleness Monitor** exists because *absence of a message is itself information* — a purely event-driven design would show a dead device as permanently healthy.

6 Technology stack (3.3)

Every row names the requirement it serves and the alternative it beat. A choice with no rejected alternative was not a choice.

Layer	Choice	Serves	Rejected, and why
Device protocol	MQTT over TLS	NFR-1, NFR-7	HTTP polling — too chatty for constrained devices
Broker	EMQX	NFR-3	Single-node Mosquitto — no clustering story
Stream	Apache Kafka	NFR-5, NFR-6	Direct DB writes — couples ingest to DB availability
Processing	Kafka Streams	NFR-1, NFR-3	Flink — more power and weight than stateless derivation needs
Current state	Redis	NFR-1, NFR-4	Latest-per-device from time-series — expensive on the hot path
Time-series	TimescaleDB	FR-9, NFR-14	InfluxDB — Timescale keeps it in Postgres, one skill set
Registry	PostgreSQL	FR-2, FR-11	Document store — data is relational and correctness matters
Read API	Go, REST+JSON	NFR-3, NFR-5	GraphQL — its strength is client-shaped queries, which we forbid
Realtime push	Server-Sent Events	FR-5	WebSocket — bidirectional, and we send nothing upstream
Auth	OIDC, short JWT	FR-11, NFR-8	Session cookies — poor fit for a static SPA on a CDN
Frontend build	Vite 5	NFR-2, NFR-12	Next.js — SSR and an API layer we do not need
Frontend UI	React 18 + TS strict	NFR-11	Svelte — smaller, but ecosystem and familiarity win
Styling	Tailwind, tokens	NFR-10, NFR-11	CSS-in-JS — runtime cost on a page that ticks live
Routing	HashRouter	NFR-12	BrowserRouter — needs server rewrites a static host cannot do
Charts	Hand-built SVG	NFR-10	Recharts/Chart.js — both fight our accessibility rules
State	React hooks	NFR-11	Redux — no shared mutable state exists to manage
Fonts	Self-hosted @fontsource	NFR-2	Google Fonts CDN — extra round trip, privacy dependency

SSE over WebSocket is the choice most worth defending. Data flows one way only. A bidirectional transport would have been a capability the requirements forbid us from using, paid for in reconnection logic.

Hand-built charts is the second. We committed to fixed-order series colours that never repaint on filtering, no dual-axis charts, and a table view on every chart. Chart libraries cycle colours by series index and own the canvas; enforcing our rules meant owning the render path.

7 Architecture design (3.4)

Style

Two styles joined at a seam. Ingest is an **event-driven pipeline**: gateway → registry check → stream → processor. Serving is a **layered client-server read model**: stores → read API → cache → static client. This is a read-optimised CQRS, and it is this simple only because the brief removed the command side.

C4 level 1 — Context

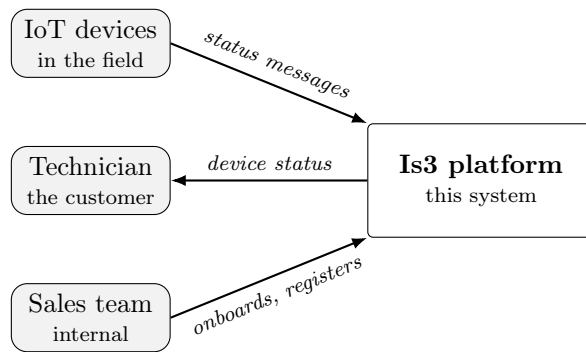


Figure 3: Context. Note what is absent: no arrow runs from the technician to the devices. There is no control path, by requirement.

C4 level 2 — Containers

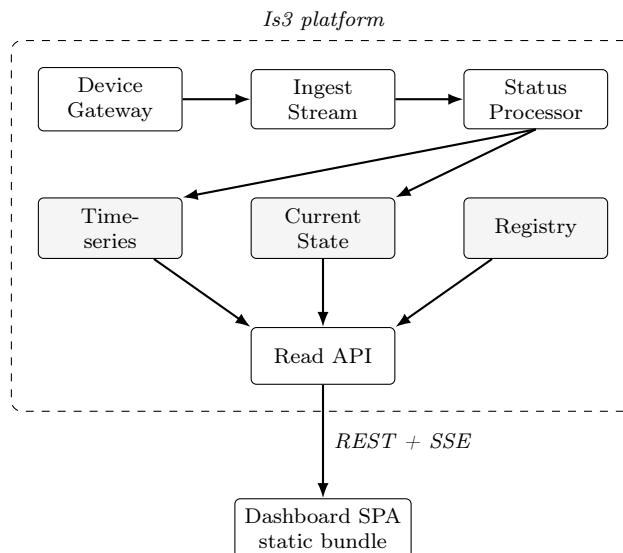


Figure 4: Containers. Arrows run one way throughout: with no user writes there is no return path, so no distributed-transaction problem exists anywhere.

C4 level 3 — Dashboard SPA

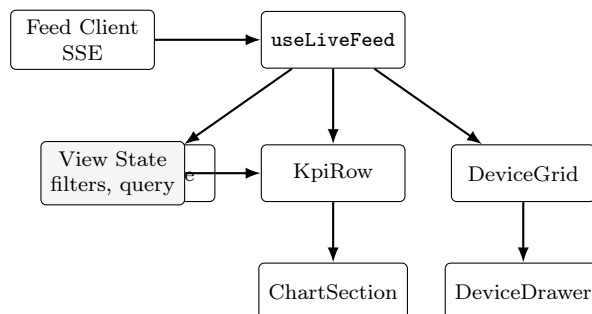


Figure 5: The frontend. `useLiveFeed` is the seam: in production it wraps the SSE connection; behind it the feed can be substituted without touching any component above.

Data flow — one way, end to end

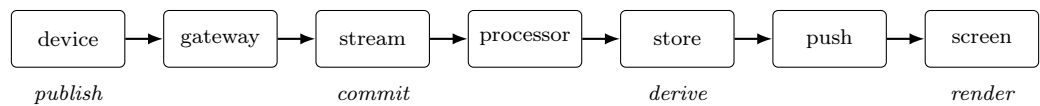


Figure 6: A reading’s journey. Budgeted against NFR-1: roughly 900 ms of work inside a 5 s allowance, leaving headroom for network variance and retry.

Development model — how the code is organised

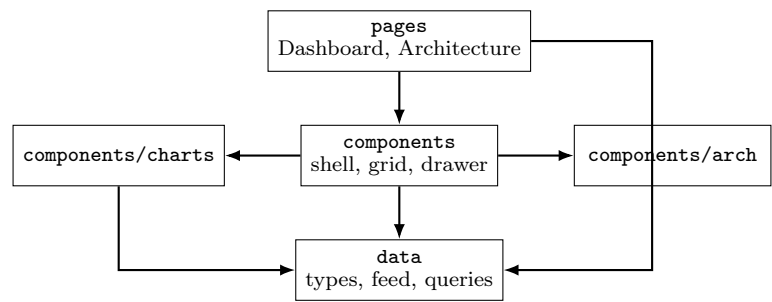


Figure 7: Package dependencies in the frontend. They point one way, downward toward **data**. Nothing in **data** imports a component, which is what keeps the mock feed swappable for a real one.

Deployment model (3.4)

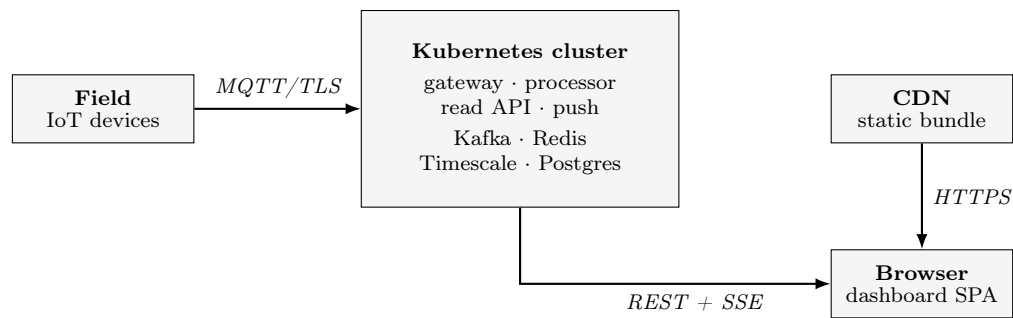


Figure 8: Deployment. The browser fetches the *application* from a CDN and the *data* from the cluster. Those are separate failure domains, which is why the display path is more available than the platform behind it (NFR-5, NFR-12).

Mapping to the 4+1 view model

The views above are not an arbitrary set. They populate Kruchten’s 4+1 model, which is the approach this course teaches for describing an architecture.

4+1 view	What it answers here	Where
Logical	Domain concepts and the components that own them	Key abstractions, 3.2
Process	Runtime elements and how a reading travels	C4 containers, data flow
Development	How the code is packaged and what depends on what	Development model
Physical	What runs where, on which failure domain	Deployment model
Scenarios	The use cases that tie the other four together	Use case diagram, 3.5

How the architecture delivers each quality attribute

Attribute	Mechanism
Latency (NFR-1)	Push over SSE, current state in memory, no polling
Read scalability (NFR-3/4)	Pre-aggregated rollups, cached responses, client-side filtering of an already-loaded fleet
Availability (NFR-5)	Static client on a CDN; ingest decoupled by the stream, so it degrades alone
Durability (NFR-6)	Durable partitioned stream with consumer offsets
Tenant isolation (NFR-8)	Tenant claim in the token, scope applied in the API, never in the client
Accessibility (NFR-10)	Status as icon + label + colour; a table view on every chart
Portability (NFR-12)	Static bundle with relative asset paths

8 Architecture decision records

ADR-001 — Separate the ingest path from the read path

Context. Devices push continuously and unpredictably; dashboards read a small set repeatedly. The brief guarantees customers never write, removing the usual reason to keep one model for both.

Decision. Devices publish into a durable partitioned stream. A processor consumes it and maintains a small current-state store plus a time-series history. The read API serves only from those stores and never touches the stream.

Consequences. Accepted: reads stay fast and cacheable regardless of ingest load; processing can fail and catch up without loss; ingest degrades alone. Given up: eventual consistency bounded by the 5 s budget in NFR-1, one more system to operate, and two stores holding overlapping data.

ADR-002 — Pre-defined queries live server-side as a catalogue

Context. FR-7 requires a fixed query set. The frontend is a static bundle, so anything it holds is public.

Decision. Each query is a server-owned definition with an id, label, description and parameterised implementation. The client sends `{queryId, params}`; the server applies the tenant scope itself, always.

Consequences. Accepted: query cost is knowable and indexable in advance; tenant scoping lives in one place; the client cannot express a query we did not plan for — which is the requirement, not a limitation. Given up: adding a query needs a backend release, and power users cannot explore freely.

ADR-003 — Charts are hand-built SVG, not a charting library

Context. We committed to fixed-order series colours, no dual-axis charts, a mandatory hover layer and a table view on every chart (NFR-10).

Decision. Small SVG components, one per chart form, sharing scale and axis helpers. Series colours are keyed by entity identity, never by array index.

Consequences. Accepted: every rule is enforceable because we own the render path; marks are real DOM nodes, so labels and focus come naturally; the chart layer costs kilobytes rather than tens of kilobytes. Given up: we implement axes, ticks, tooltips and legends ourselves, and a newcomer must learn our API rather than a familiar one.

9 Known risks

- **The stream is the component whose loss hurts most.** Partitioning and replication mitigate it, but it remains a single conceptual point of failure.

- **Staleness detection depends on correct per-device expected intervals.** A wrong interval means either false offline alerts or silent dead devices. This needs to be a monitored data-quality metric, not a set-and-forget field.
- **Client-side filtering assumes the fleet fits in the browser.** True at 10 000 devices with a compact representation; beyond that, filtering moves server-side.